

Proyecto Estratégico “Madurez, Adaptabilidad, Resiliencia y Continuidad en Organizaciones Digitalizadas. MARCO Diligencia)”: Proyecto **Incibepedia**

(Periodo septiembre 2025 – diciembre 2025)

Ares Alfayate Santiago

Bermejo Fernádnuez César

Díez Tejedor Gabriel

Índice

1. Introducción.....	3
2. Análisis de las plataformas evaluadas.....	3
Confluence.....	3
MediaWiki.....	3
BookStack.....	4
Outline.....	4
DokuWiki.....	4
3. Comparativa general y criterios de evaluación.....	5
4. Justificación de la elección de Wiki.js.....	6
5. Trabajo realizado.....	6
6. Guía de uso.....	11
Cambio de idioma.....	11
Navegación y comentarios.....	11
Creación y edición de archivos.....	12
Búsqueda por tema.....	12
7. Scripts.....	12
Primera versión para probar la sustitución de texto para las referencias de Markdown utilizando un programa llamado grepWin.....	13
Primera versión utilizando BeautifulSoup.....	13
Versión final utilizando html2text.....	14
Procesamiento de los archivos Markdown.....	15
Eliminación de los paréntesis en los ficheros para evitar compatibilidad con WikiJS.....	17
Eliminación de las últimas líneas que no contienen información.....	18
Añadir metadatos.....	19
Traducción a Español.....	20
Corrección del formato del bloque de metadatos.....	23
Traspaso de la ruta de los archivos dentro de la wiki.....	25

1. Introducción

Durante la fase inicial del proyecto se llevó a cabo un proceso de análisis y deliberación con el objetivo de seleccionar la plataforma más adecuada para la gestión, organización y publicación de la información recopilada. Dado el carácter documental y colaborativo del proyecto, resultaba imprescindible evaluar distintas soluciones wiki ampliamente utilizadas, teniendo en cuenta aspectos como la facilidad de uso, la flexibilidad, la escalabilidad, la seguridad y los requisitos técnicos de implementación.

En este contexto, se analizaron diversas plataformas consolidadas en el ámbito de la documentación colaborativa, concretamente **Confluence**, **MediaWiki**, **BookStack**, **Outline** y **DokuWiki**, valorando sus ventajas, limitaciones y grado de adecuación a los objetivos y restricciones del proyecto.

2. Análisis de las plataformas evaluadas

Confluence

Confluence es una plataforma empresarial diseñada para centralizar el conocimiento interno, ofreciendo herramientas avanzadas de colaboración y gestión documental. Su editor es potente, flexible y permite a los equipos crear contenidos estructurados con facilidad, además de ofrecer un control granular de permisos, integraciones con Jira y un ecosistema muy robusto.

Su fortaleza radica en la capacidad de crear espacios de trabajo modernos y muy organizados, ideales para empresas medianas o grandes que buscan estandarizar procesos y mantener una documentación viva.

El inconveniente principal es su coste y complejidad. Aunque la versión Cloud funciona muy bien, el modelo Data Center es caro y requiere infraestructura adicional. Esto lo convierte en una opción excelente para organizaciones empresariales, pero menos adecuada para equipos pequeños o proyectos que buscan una solución ligera y completamente autogestionada.

MediaWiki

MediaWiki es el motor detrás de Wikipedia y está pensado para gestionar grandes volúmenes de información de forma estructurada, duradera y muy escalable. Su arquitectura está diseñada para wikis inmensos y contiene un ecosistema de miles de extensiones que permiten añadir semántica, plantillas avanzadas y automatización del contenido.

Es una herramienta ideal para crear enciclopedias internas complejas con multitud de artículos conectados entre sí.

Por otro lado, su curva de aprendizaje es notablemente más alta. El editor visual ha mejorado con los años, pero sigue siendo más técnico que otras alternativas. También requiere más

configuración inicial si buscas un entorno moderno. Aun así, su solidez lo convierte en una opción excepcional para proyectos donde prima la estructura y la escalabilidad.

BookStack

BookStack apuesta por la simplicidad y la claridad, organizando el conocimiento en una jerarquía limpia basada en libros, capítulos y páginas. Esto lo hace extremadamente intuitivo, ideal para equipos que buscan estructurar documentación rápidamente sin necesidad de aprender complejos sistemas de plantillas o extensiones.

Su editor WYSIWYG es muy fácil de usar y su instalación autohospedada es ligera y sencilla.

Sin embargo, su estructura rígida también puede ser una limitación para casos donde se desea un enfoque más libre o tipo enciclopedia. BookStack destaca en documentación, manuales y procedimientos, pero no está tan orientado a artículos extensos como en un wiki tradicional. Aun así, es una alternativa muy equilibrada para documentación interna.

Outline

Outline ofrece una experiencia moderna, minimalista y muy similar a Notion, pero completamente autohospedable. Está pensado para equipos que quieren una interfaz limpia y una colaboración fluida sin complicaciones. Utiliza Markdown como base, integra permisos por equipos y se conecta fácilmente con Slack, almacenamiento externo y SSO empresarial.

Es rápido, ligero y muy amigable para usuarios no técnicos.

Su limitación principal es que es menos enciclopédico y más colaborativo, centrado en la edición de documentos en lugar de en un sistema wiki clásico. Aunque su estructura es potente, no está pensado para wikis extremadamente grandes o con necesidades semánticas. Aun así, la documentación dinámica es una solución muy moderna.

DokuWiki

DokuWiki es una solución wiki ligera, sin base de datos, que almacena todo en archivos planos. Esto lo convierte en una herramienta extremadamente fácil de mantener y migrar, ideal para organizaciones que buscan simplicidad y fiabilidad. Su sistema de plugins es amplio y permite adaptarlo a muchas necesidades.

Además, es muy estable, rápido y funciona bien incluso con recursos limitados.

La contrapartida es que su interfaz y experiencia visual son menos modernas que otras alternativas como Wiki.js o Outline. Aunque puede mejorarse con temas y plugins, requiere más personalización para lograr un look contemporáneo. No obstante, sigue siendo una solución sólida y sencilla para wikis internos.

3. Comparativa general y criterios de evaluación

Durante el proceso de evaluación se tuvieron en cuenta los siguientes criterios clave:

- Facilidad de uso y experiencia de usuario.
- Flexibilidad en la organización del contenido.
- Capacidad de control de accesos y permisos.
- Escalabilidad y mantenimiento a largo plazo.
- Naturaleza abierta y posibilidad de autoalojamiento.
- Compatibilidad con flujos de trabajo modernos (Markdown, Git, APIs).

Desde esta perspectiva, se observó que ninguna de las plataformas analizadas resulta deficiente en términos absolutos, sino que cada una está orientada a necesidades y contextos específicos.

Plataforma	Licencia	Facilidad de uso	Flexibilidad	Autoalojamiento	Adecuación al proyecto
Confluence	Propietaria	Alta	Media	Limitado	Media
MediaWiki	Código abierto	Baja	Alta	Sí	Media
BookStack	Código abierto	Alta	Media-baja	Sí	Media
Outline	Código abierto	Alta	Media	Complejo	Media-baja
DokuWiki	Código abierto	Media	Baja	Sí	Baja
Wiki.js	Código abierto	Alta	Alta	Sí	Alta

4. Justificación de la elección de Wiki.js

Tras el análisis comparativo de las diferentes alternativas, se optó finalmente por **Wiki.js** como plataforma base del proyecto. Esta decisión se fundamenta en una combinación equilibrada de factores técnicos, funcionales y operativos.

Wiki.js ofrece una **interfaz moderna y accesible**, que facilita la creación y edición de contenidos tanto a usuarios técnicos como no técnicos, gracias a su soporte nativo para **Markdown** y a la disponibilidad de editores visuales. Asimismo, incorpora un **sistema avanzado de gestión de usuarios y permisos**, lo que permite definir con precisión los niveles de acceso y garantizar la seguridad del contenido.

Desde el punto de vista técnico, su arquitectura basada en **Node.js** y su compatibilidad con múltiples bases de datos relacionales proporcionan una solución flexible y escalable, adecuada para el crecimiento progresivo del proyecto. La posibilidad de integración con **repositorios Git** aporta, además, un valor añadido en términos de control de versiones, trazabilidad de cambios y respaldo de la información.

Otro aspecto determinante ha sido su condición de **software de código abierto**, que garantiza independencia tecnológica, transparencia y capacidad de adaptación a necesidades futuras sin depender de soluciones propietarias. Todo ello, unido a una documentación clara y a una comunidad activa, convierte a Wiki.js en una plataforma especialmente adecuada para proyectos que requieren un equilibrio entre potencia, usabilidad y control.

El proceso de selección de la plataforma wiki se ha basado en un análisis riguroso de las principales alternativas disponibles, teniendo en cuenta tanto sus fortalezas como sus limitaciones. Aunque todas las soluciones evaluadas presentan ventajas relevantes en determinados contextos, **Wiki.js** destaca por ofrecer una combinación óptima de modernidad, flexibilidad, facilidad de uso y control técnico, lo que la convierte en la opción más adecuada para los objetivos y requisitos del presente proyecto.

5. Trabajo realizado

Para la recopilación y consolidación de la información empleada en el desarrollo del presente proyecto, se ha llevado a cabo un proceso metódico y planificado de **rastreo automatizado (crawling) y extracción de contenidos** procedentes de un conjunto seleccionado de recursos digitales especializados. Estas fuentes están constituidas, fundamentalmente, por **diccionarios de vocabulario, glosarios terminológicos sectoriales y plataformas de información reconocidas por su fiabilidad y rigor**, lo que ha permitido asegurar un elevado nivel de calidad y consistencia en los datos recopilados.

La selección de las fuentes de información se ha realizado siguiendo criterios estrictos de **relevancia temática, autoridad de la fuente, actualización de los contenidos y estructura de la información**, priorizando aquellos recursos que presentan una organización clara y sistemática,

lo cual facilita su procesamiento automatizado. Este enfoque ha permitido optimizar los procesos de crawling, reducir la aparición de datos redundantes o inconsistentes y garantizar una extracción homogénea de la información, minimizando la necesidad de intervenciones manuales posteriores.

El volumen de datos obtenido ha sido cuidadosamente calibrado con el objetivo de alcanzar un **equilibrio óptimo entre exhaustividad y viabilidad operativa**. En este sentido, se ha considerado esencial recopilar una cantidad de información lo suficientemente amplia como para ofrecer una **base de conocimiento representativa y funcional** que permita validar el planteamiento inicial del proyecto y demostrar su potencial en una primera fase de implementación. Al mismo tiempo, se ha limitado deliberadamente la magnitud del conjunto de datos para evitar una sobrecarga informativa que pudiera afectar negativamente al rendimiento de los sistemas implicados, a los tiempos de procesamiento o a la estabilidad general de la infraestructura técnica empleada.

Asimismo, durante el proceso de recopilación se han aplicado mecanismos de **control y filtrado de datos**, destinados a descartar contenidos irrelevantes, duplicados o carentes de valor informativo para los objetivos del proyecto. Este tratamiento preliminar de la información resulta clave para garantizar que el conjunto de datos final sea coherente, estructurado y directamente utilizable en las fases posteriores de análisis, integración y explotación.

Cabe destacar que esta estrategia de recopilación no persigue únicamente satisfacer las necesidades inmediatas del proyecto, sino que ha sido concebida con una **visión a medio y largo plazo**, permitiendo una futura ampliación progresiva del volumen y diversidad de los datos sin comprometer la escalabilidad del sistema. De este modo, el conjunto de datos inicial actúa como un **entorno de prueba controlado**, idóneo para evaluar el funcionamiento de las herramientas desarrolladas, detectar posibles limitaciones técnicas y definir mejoras iterativas basadas en resultados empíricos.

En conclusión, la metodología adoptada para la recopilación de la información se fundamenta en un enfoque equilibrado, riguroso y sostenible, que prioriza tanto la **calidad y fiabilidad de los datos** como la **eficiencia operativa y la viabilidad técnica** del proyecto. Este planteamiento garantiza una base sólida sobre la que construir desarrollos posteriores, facilitando la evolución del sistema de forma controlada y coherente con los objetivos establecidos.

Tras la extracción inicial de las páginas relevantes en formato HTML, se identificó la necesidad de realizar un proceso de transformación y adecuación del contenido antes de su integración en la wiki corporativa. Para ello, se diseñaron y desarrollaron varios scripts específicos cuyo objetivo principal fue convertir el contenido desde HTML a Markdown. La elección de Markdown como formato de destino respondió a criterios tanto técnicos como operativos, ya que este lenguaje reduce de manera considerable la complejidad asociada a la gestión de múltiples etiquetas y estructuras propias de HTML. Asimismo, facilita la edición y el mantenimiento del contenido por parte de usuarios sin conocimientos técnicos avanzados, promoviendo así una mayor autonomía y eficiencia en la actualización de la documentación.

El proceso de conversión y normalización del contenido se estructuró en una serie de fases claramente definidas, que permitieron garantizar la coherencia, compatibilidad y calidad de la información publicada:

En primer lugar, se procedió a la conversión de todas las referencias, etiquetas y estructuras propias de HTML a su equivalente en Markdown. Esta tarea incluyó la adaptación de encabezados, listas, tablas, enlaces e imágenes, asegurando que el contenido mantuviera su legibilidad y estructura original, al tiempo que se ajustaba a las limitaciones y ventajas del nuevo formato.

A continuación, se realizó una revisión exhaustiva de los enlaces internos presentes en las páginas. Estos enlaces fueron modificados para que apuntaran exclusivamente a páginas alojadas dentro de la propia wiki, evitando dependencias externas y posibles errores derivados de cambios en sitios de terceros. Este paso fue clave para garantizar la integridad de la navegación interna y la correcta interconexión entre los distintos artículos.

Posteriormente, se eliminaron todos aquellos bloques de texto y elementos visuales que no aportan información relevante al contenido documental. Entre estos elementos se incluyeron encabezados generales, pies de página, menús de navegación y otros componentes propios de la estructura del sitio web original del cual se extrajo la información. Esta depuración permitió centrar el contenido únicamente en la información útil y relevante para el usuario final.

Seguidamente, se revisaron y corrigieron los enlaces dirigidos a las secciones de referencias bibliográficas o documentales, con el fin de asegurar su correcto funcionamiento tras el cambio de formato y estructura. Este ajuste fue necesario para mantener la trazabilidad de las fuentes y la validez de las referencias incluidas en cada página.

Otro aspecto relevante del proceso fue la adaptación de los nombres de archivo. Se detectó que algunos de ellos contenían paréntesis, caracteres que no son admitidos por la plataforma Wiki.js. Por este motivo, se llevó a cabo un proceso de renombrado sistemático de los archivos, sustituyendo dichos caracteres por alternativas compatibles, garantizando así su correcta carga y visualización dentro del sistema.

En relación con la internacionalización del contenido, se procedió a la traducción de las páginas manteniendo inalteradas tanto las referencias como los enlaces internos. Esta restricción fue fundamental para evitar la ruptura de enlaces provocada por cambios en los nombres de los archivos o identificadores. La traducción se realizó modificando únicamente la configuración regional (locale) de la web, lo que permitió ofrecer el contenido en el idioma deseado sin comprometer la integridad estructural de la wiki.

Finalmente, se incorporó un bloque de metadatos en cada una de las páginas generadas. Dicho bloque permite a la plataforma identificar y gestionar correctamente información clave como el título de la página, el tipo de contenido, la fecha de la última modificación y el estado de publicación. Esta información resulta esencial para el correcto funcionamiento de la wiki, así como para facilitar tareas de mantenimiento, versionado y despliegue del contenido.

En conjunto, este proceso integral de conversión, adaptación y enriquecimiento del contenido aseguró una transición ordenada y eficiente hacia la nueva plataforma, mejorando la accesibilidad, mantenibilidad y calidad general de la documentación disponible en la wiki.

Para desplegar el servidor de la wiki y el servidor de datos correspondiente, se ha optado por elegir Docker conjunto con un servidor PostgreSQL.

El uso de Docker permite encapsular aplicaciones y sus dependencias dentro de contenedores aislados, lo que asegura consistencia entre entornos de desarrollo, pruebas y producción. En este contexto, Wiki.js se ejecuta como una aplicación web contenida que requiere una base de datos relacional para almacenar su información. PostgreSQL se presenta como una opción robusta y ampliamente utilizada para este propósito. Para reforzar la seguridad, las credenciales necesarias para la conexión entre ambos servicios no se definen de forma explícita en archivos de configuración, sino que se gestionan mediante Docker Secrets.

La arquitectura planteada se basa en dos contenedores principales que se comunican a través de una red interna de Docker. Wiki.js actúa como servicio frontal, mientras que PostgreSQL funciona como backend de persistencia. Docker Secrets se encarga de almacenar de forma cifrada los datos sensibles, tales como el usuario de la base de datos, la contraseña y el nombre de la base de datos, limitando su acceso únicamente a los servicios autorizados. Para que esta funcionalidad esté disponible, es necesario trabajar en modo Docker Swarm, el cual debe inicializarse previamente en el sistema anfitrión.

Antes de desplegar los servicios, se procede a la creación de los secretos. Este proceso se realiza directamente desde la línea de comandos, evitando que las credenciales queden registradas en texto plano en archivos del sistema. Los secretos se crean de la siguiente manera:

Shell

```
echo "wikijs_db_user" | docker secret create postgres_user -  
echo "contraseña_segura_db" | docker secret create postgres_password -  
echo "wikijs_db" | docker secret create postgres_db -
```

Una vez creados los secretos, estos quedan almacenados de forma segura dentro del motor de Docker y solo pueden ser consumidos por los servicios que se definan explícitamente en el archivo de orquestación. Para el despliegue de Wiki.js y PostgreSQL se utiliza un archivo **docker-compose.yml** compatible con Docker Swarm, en el cual se describen ambos servicios y su relación con los secretos previamente creados.

None

```
version: "3.8"  
services:  
  postgres:  
    image: postgres:15  
    environment:  
      POSTGRES_USER_FILE: /run/secrets/postgres_user  
      POSTGRES_PASSWORD_FILE: /run/secrets/postgres_password  
      POSTGRES_DB_FILE: /run/secrets/postgres_db  
    secrets:  
      - postgres_user  
      - postgres_password  
      - postgres_db
```

```
volumes:
  - postgres_data:/var/lib/postgresql/data
deploy:
  restart_policy:
    condition: on-failure

wikijs:
  image: ghcr.io/requarks/wiki:2
  depends_on:
    - postgres
  environment:
    DB_TYPE: postgres
    DB_HOST: postgres
    DB_PORT: 5432
    DB_USER_FILE: /run/secrets/postgres_user
    DB_PASS_FILE: /run/secrets/postgres_password
    DB_NAME_FILE: /run/secrets/postgres_db
  secrets:
    - postgres_user
    - postgres_password
    - postgres_db
  ports:
    - "3000:3000"
  deploy:
    restart_policy:
      condition: on-failure

secrets:
  postgres_user:
    external: true
  postgres_password:
    external: true
  postgres_db:
    external: true

volumes:
  postgres_data:
```

En este archivo se observa que ni Wiki.js ni PostgreSQL reciben directamente valores sensibles como variables de entorno tradicionales. En su lugar, se utilizan variables con el sufijo **_FILE**, las cuales indican a la aplicación que debe leer el contenido de un archivo. Dichos archivos son proporcionados automáticamente por Docker en la ruta **/run/secrets/**, donde cada secreto se monta como un archivo de solo lectura. Esta técnica evita la exposición de credenciales en texto plano, tanto en el archivo de configuración como en los logs del sistema.

El despliegue del conjunto de servicios se realiza mediante el comando de Docker Stack, el cual permite lanzar el stack completo dentro del clúster Swarm de forma controlada:

Shell

```
docker stack deploy -c docker-compose.yml wikijs_stack
```

Una vez ejecutado este comando, Docker crea los servicios correspondientes y gestiona su ciclo de vida. Wiki.js queda disponible a través del puerto 3000 del servidor anfitrión y, al acceder por primera vez, se presenta el asistente de configuración inicial. En este proceso, la aplicación ya dispone de la información necesaria para conectarse a PostgreSQL, ya que las credenciales han sido suministradas de forma segura mediante los secretos.

Este enfoque proporciona una solución alineada con buenas prácticas de seguridad y despliegue. La separación clara de servicios, la persistencia de datos mediante volúmenes y la protección de información sensible mediante Docker Secrets hacen que la arquitectura sea adecuada para entornos productivos. Además, la solución es fácilmente escalable y mantenible, lo que facilita su administración a largo plazo.

En conclusión, el uso de Docker junto con Docker Swarm y Docker Secrets permite desplegar Wiki.js y PostgreSQL de forma segura, ordenada y profesional. Este método reduce significativamente el riesgo de exposición de credenciales y proporciona una base sólida para la operación de servicios de documentación en entornos empresariales o académicos.

6. Guía de uso

Cambio de idioma

Procedimiento para cambiar el idioma de la interfaz web de la wiki. En la grabación se aprecia cómo el usuario accede al menú del idioma y selecciona el idioma deseado, tras lo cual la plataforma actualiza inmediatamente los textos de la interfaz. Este proceso demuestra la flexibilidad del sistema y la sencillez con la que se pueden adaptar los contenidos a diferentes idiomas.

Navegación y comentarios

Navegación por el índice de la web de la wiki, donde el usuario selecciona distintas páginas para acceder a su contenido. Asimismo, se muestra la funcionalidad de comentarios, evidenciando cómo se añade un comentario en una página, se navega posteriormente a otra sección y se

intenta publicar un nuevo comentario. Aquí se presenta el aviso del sistema que informa sobre la superación del límite de tiempo establecido para realizar varios comentarios consecutivos, lo que pone de manifiesto los mecanismos de control y moderación integrados en la plataforma.

Creación y edición de archivos

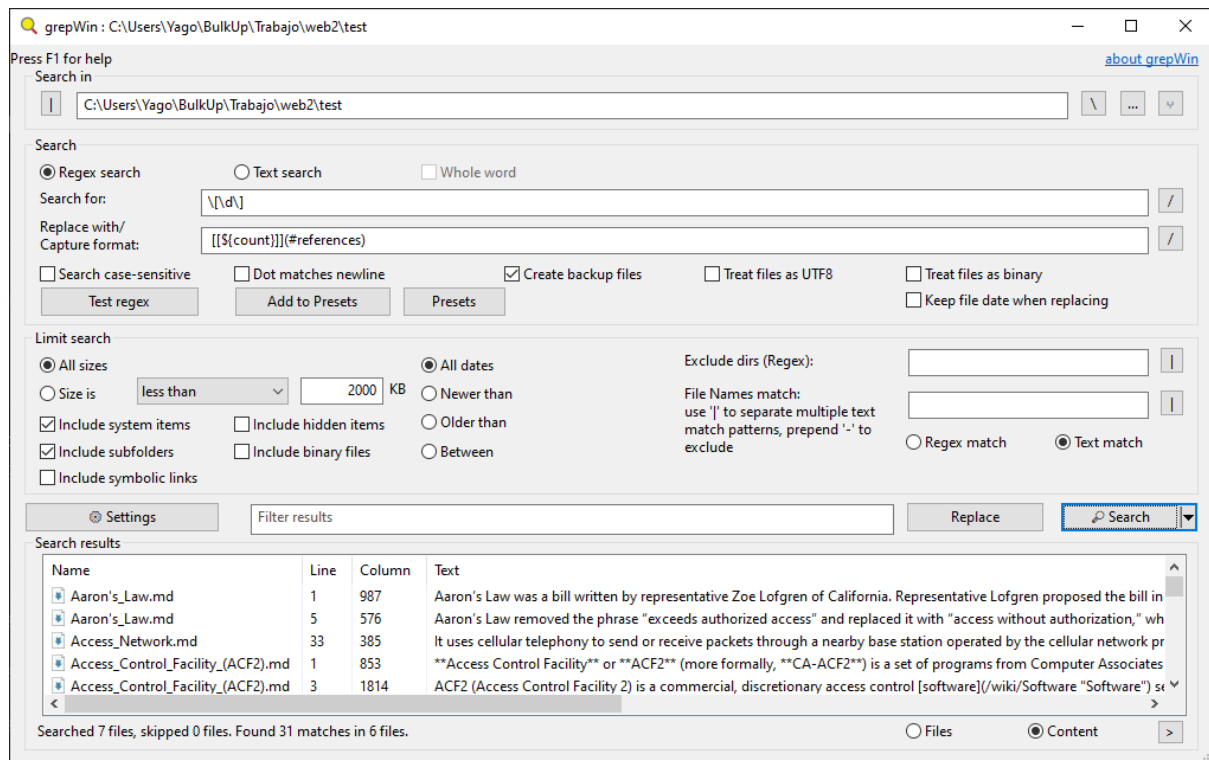
Aquí se observa cómo un usuario autorizado edita una página existente de la wiki y guarda los cambios para que se reflejen de inmediato en la plataforma. Posteriormente, se muestra el proceso de creación de una nueva página, en el que se define la ruta del archivo que almacenará la información y se selecciona el tipo de archivo a utilizar, como Markdown o HTML. Finalmente, el video presenta las opciones de configuración de la página, donde se establecen el título, los tags para agrupar el contenido, la ruta definitiva del archivo y una breve descripción, evidenciando las herramientas disponibles para una gestión organizada y estructurada de la información.

Búsqueda por tema

Funcionamiento del sistema de búsquedas de la web, donde el usuario introduce un término y la plataforma devuelve resultados basados tanto en el título de las páginas como en el nombre del archivo que contiene la información. Asimismo, se aprecia que junto a cada resultado se indica el idioma de la página a la que se va a acceder, lo que facilita la identificación del contenido y mejora la experiencia de navegación dentro de la wiki.

7. Scripts

Primera versión para probar la sustitución de texto para las referencias de Markdown utilizando un programa llamado grepWin



Primera versión utilizando BeautifulSoup

El primer script desarrollado utiliza la librería BeautifulSoup para realizar una conversión inicial y básica de los archivos HTML a formato Markdown. El código funciona cargando el contenido del archivo y utilizando un parseador que identifica las etiquetas de enlace; mediante un bucle, el script localiza cada etiqueta de hipervínculo y la sustituye manualmente por la sintaxis de corchetes y paréntesis propia de Markdown. Finalmente, emplea una función de extracción de texto con un separador de salto de línea para obtener un documento limpio, eliminando el resto del etiquetado HTML original pero conservando los enlaces ya convertidos.

```
Python
import os
from bs4 import BeautifulSoup
```

```
# --- Configuration ---
input_folder = "backup"      # Folder containing .html files
output_folder = "markdown"   # Where to save .md files

# Create output folder if it doesn't exist
os.makedirs(output_folder, exist_ok=True)

# Loop through all HTML files in the folder
for filename in os.listdir(input_folder):
    if filename.lower().endswith(".html"):
        input_path = os.path.join(input_folder, filename)
        output_path = os.path.join(output_folder, os.path.splitext(filename)[0] +
        ".md")

        # Read HTML content
        with open(input_path, "r", encoding="utf-8") as file:
            html = file.read()

        # Parse HTML
        soup = BeautifulSoup(html, "html.parser")

        # Convert <a> tags into Markdown-style links
        for a in soup.find_all('a', href=True):
            markdown_link = f"[{a.get_text(strip=True)}]({a['href']})"
            a.replace_with(markdown_link)

        # Extract readable text (including replaced markdown links)
        markdown_text = soup.get_text(separator="\n", strip=True)

        # Write to Markdown file
        with open(output_path, "w", encoding="utf-8") as md_file:
            md_file.write(markdown_text)

        print(f"Converted: {filename} → {os.path.basename(output_path)}")

print("\nAll HTML files converted to Markdown!")
```

Versión final utilizando html2text

Para mejorar la fidelidad del formato, se implementó una segunda versión basada en la librería **html2text**. En este caso, el script configura un objeto conversor al que se le asignan parámetros específicos, como la desactivación del ancho de línea automático para evitar saltos innecesarios y la instrucción de no ignorar los enlaces. Al procesar el HTML a través de este motor especializado, se obtiene un resultado mucho más estructurado y profesional que el método anterior, respetando mejor la jerarquía visual del contenido original.

```
Python
import os
import html2text

# --- Configuration ---
input_folder = "html"      # Folder containing .html files
output_folder = "markdown" # Folder for output .md files

# Create output folder if it doesn't exist
os.makedirs(output_folder, exist_ok=True)

# Initialize html2text converter
converter = html2text.HTML2Text()
converter.ignore_links = False # Keep [text](link)
converter.ignore_images = True
converter.body_width = 0      # Prevent wrapping lines

# Loop through all HTML files in the folder
for filename in os.listdir(input_folder):
    if filename.lower().endswith(".html"):
        input_path = os.path.join(input_folder, filename)
        output_path = os.path.join(output_folder, os.path.splitext(filename)[0] +
                                    ".md")

        with open(input_path, "r", encoding="utf-8") as f:
            html = f.read()
        markdown_text = converter.handle(html)
        with open(output_path, "w", encoding="utf-8") as f:
            f.write(markdown_text)

        print(f"Converted: {filename} → {os.path.basename(output_path)}")

print("\nAll HTML files converted to Markdown with formatting!")
```

Procesamiento de los archivos Markdown

Una vez obtenido el Markdown, se aplica un script de procesamiento y limpieza que utiliza **expresiones regulares (regex)** para pulir el contenido. El código busca patrones numéricos de referencias bibliográficas y los transforma en enlaces internos que apuntan a una sección de "referencias" al final del documento. Además, este script incluye una lógica de filtrado que recorre las líneas del archivo para detectar y eliminar bloques de navegación innecesarios del sitio de origen, deteniendo la lectura por completo cuando encuentra la frase "Retrieved from", lo que permite descartar automáticamente toda la información de pie de página residual.

Python

```
import os
import re

def procesar_markdowns(carpeta):
    # Reemplazo de referencias [n]
    patron_refs = re.compile(r"\[d\]")

    # Reemplazo de /wiki/...\( ... \)
    patron_wiki = re.compile(r"( /wiki/[^\s]+?)\\((.+?)\\)")

    # Detecta líneas tipo: [ ](/wiki/....)
    patron_linea_wiki = re.compile(r'^\[ \]\( /wiki/.+\'')

    # Detecta inicio del bloque final
    patron_retrieved = re.compile(r'^Retrieved from')

    carpeta_salida = os.path.join(carpeta, "fix")
    os.makedirs(carpeta_salida, exist_ok=True)

    for archivo in os.listdir(carpeta):
        if archivo.endswith(".md"):
            ruta_entrada = os.path.join(carpeta, archivo)
            ruta_salida = os.path.join(carpeta_salida, archivo)

            with open(ruta_entrada, "r", encoding="utf-8") as f:
                lineas = f.readlines()

            contenido = "".join(lineas)

            # ----- Reemplazo 1: [n] -----
            contador = 1
            def reemplazo_refs(match):
                nonlocal contador
                nuevo = f" \[{contador}\] (#references)"
                contador += 1
                return nuevo

            contenido = patron_refs.sub(reemplazo_refs, contenido)

            # ----- Reemplazo 2: /wiki/...\( ... \) -----
            contenido = patron_wiki.sub(r"\1\2", contenido)

            # ----- Eliminación de líneas wiki + siguiente -----
            lineas = contenido.split("\n")
            nuevas_lineas = []
            i = 0
```



```
while i < len(lineas):
    linea = lineas[i]

    if patron_linea_wiki.match(linea.strip()):
        # borrar esa y la siguiente
        i += 2
        continue

    nuevas_lineas.append(linea)
    i += 1

lineas = nuevas_lineas
nuevas_lineas = []

# ----- Eliminación del bloque final -----
for linea in lineas:
    if patron_retrieved.match(linea.strip()):
        break # terminamos aquí, nada más se copia
    nuevas_lineas.append(linea)

contenido_final = "\n".join(nuevas_lineas)

# Guardar archivo corregido
with open(ruta_salida, "w", encoding="utf-8") as f:
    f.write(contenido_final)

print(f"Procesado: {archivo} → {ruta_salida}")

if __name__ == "__main__":
    carpeta = "." # Carpeta donde se hara el trabajo
    procesar_markdowns(carpeta)
```

Eliminación de los paréntesis en los ficheros para evitar compatibilidad con WikiJS

Debido a las restricciones técnicas de **Wiki.js**, que no permite paréntesis en los nombres de los archivos, se creó un script específico para el renombrado masivo de ficheros. El código recorre el directorio de trabajo, identifica los caracteres de apertura y cierre de paréntesis en los nombres de archivo y los elimina mediante una función de reemplazo de cadenas. Posteriormente, utiliza una librería de gestión de archivos para copiar los documentos resultantes en una nueva carpeta, garantizando que todos los nombres sean compatibles con la plataforma de destino sin perder la integridad de los datos.

Python

```
import os
import shutil

def rename_files_no_parenthesis(carpeta):
    # Crear la carpeta de salida si no existe
    carpeta_salida = os.path.join(carpeta, "No_parenthesis")
    os.makedirs(carpeta_salida, exist_ok=True)

    for filename in os.listdir(carpeta):
        if filename.endswith(".md"):
            # Eliminar parentesis del nombre
            nueva_nombre = filename.replace("(", "").replace(")", "")

            ruta_origen = os.path.join(carpeta, filename)
            ruta_destino = os.path.join(carpeta_salida, nueva_nombre)

            # Copiar el archivo al nuevo destino
            shutil.copy2(ruta_origen, ruta_destino)
            print(f"Copied: {filename} → {nueva_nombre}")

if __name__ == "__main__":
    carpeta = "." # Carpeta donde se hara el trabajo
    rename_files_no_parenthesis(carpeta)
```

Eliminación de las últimas líneas que no contienen información

En una fase previa del tratamiento de datos, se utilizó un script diseñado para eliminar las líneas de cabecera que no contenían información útil en los archivos HTML originales. Este código funciona leyendo el archivo completo como una lista de líneas y aplicando un recorte directo que descarta las primeras 111 líneas, donde se concentraba el código de navegación y metadatos de la web antigua. Al reescribir el archivo empezando desde la línea 112, se asegura que el conversor posterior trabaje únicamente con el cuerpo principal del artículo.

Python

```
import os

folder = "C:/Users/Yago/BulkUp/Trabajo/Incibepedia"
```

```
for filename in os.listdir(folder):
    if filename.endswith(".html"):
        file_path = os.path.join(folder, filename)
        with open(file_path, "r", encoding="utf-8") as f:
            lines = f.readlines()
        with open(file_path, "w", encoding="utf-8") as f:
            f.writelines(lines[111:])
```

Añadir metadatos

Para que la wiki reconozca correctamente las páginas, un script se encarga de añadir el bloque de **metadatos o Front Matter** al inicio de cada archivo Markdown. El código extrae el nombre del archivo, limpia los guiones bajos para convertirlo en un título legible y construye una cabecera en formato YAML que incluye la descripción, la fecha de publicación y las etiquetas. Antes de insertar este bloque, el script realiza una comprobación lógica para verificar si el archivo ya tiene metadatos, evitando así duplicaciones accidentales en caso de que el proceso se ejecute varias veces.

```
Python
from pathlib import Path
from datetime import datetime

INPUT_DIR = Path("wiki")
OUTPUT_DIR = Path("wiki_en")
COUNTER_SKIPPED = 0
COUNTER_PROCESSED = 0
FILE_SKIPPED_LIST = [] # Add filenames to skip if needed

def add_title():
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    today = datetime.today().strftime("%Y-%m-%d")

    for file_path in INPUT_DIR.iterdir():
        if not file_path.is_file():
            continue

        title = file_path.stem.replace("_", " ")

        front_matter = f"""---
title: {title}
```

```
description:
published: true
date: {today}
tags:
editor: markdown
dateCreated: {today}
---

"""

content = file_path.read_text(encoding="utf-8")

# Skip if front matter already exists
if content.lstrip().startswith("---"):
    global COUNTER_SKIPPED
    COUNTER_SKIPPED += 1
    FILE_SKIPPED_LIST.append(file_path.name)
    print(f"Skipping (already has front matter): {file_path.name}")
    output_content = content
else:
    global COUNTER_PROCESSED
    COUNTER_PROCESSED += 1
    output_content = front_matter + content

output_path = OUTPUT_DIR / file_path.name
output_path.write_text(output_content, encoding="utf-8")

print(f"Processed: {file_path.name}")

print(f"\nSummary:")
print(f"Processed files: {COUNTER_PROCESSED}")
print(f"Skipped files: {COUNTER_SKIPPED}")
print(f"Skipped file names: {FILE_SKIPPED_LIST}")

if __name__ == "__main__":
    add_title()
```

Traducción a Español

La traducción del contenido se gestiona mediante un script que integra la **API de Google Gemini**. El código configura el modelo de inteligencia artificial con instrucciones precisas para traducir el texto del inglés al español, advirtiéndolo explícitamente a la IA que no debe modificar ni las rutas de los archivos ni el bloque de metadatos inicial para no romper la funcionalidad de la wiki. Además,

el script incluye un control de tiempo que introduce una pausa de un segundo entre la traducción de cada archivo, lo que permite cumplir con los límites de peticiones de la API sin interrumpir el flujo de trabajo.

```
Python
import os
import time
import google.generativeai as genai

# ===== CONFIGURATION =====
# 1. Paste your API Key here
API_KEY = "YOUR_API_KEY_HERE"

# 2. Setup your languages
TARGET_LANGUAGE = "Spanish" # Change this to your desired language

# 3. Folder paths (Make sure these exist!)
INPUT_FOLDER = "en/wiki"
OUTPUT_FOLDER = "wiki_es"

# 4. Model Selection
# 'gemini-1.5-flash' is faster and has higher rate limits.
# 'gemini-1.5-pro' is smarter but has stricter rate limits.
MODEL_NAME = "gemini-2.5-flash"

# 5. Safety Delay (Seconds)
# Free tier allows roughly 15 requests per minute.
# 4 seconds is the mathematical limit, 5 is safe.
DELAY_BETWEEN_FILES = 1
# =====

def setup_gemini():
    genai.configure(api_key=API_KEY)
    # Configure safety settings to reduce false refusals
    # (Note: severe safety violations will still be blocked by Google)
    safety_settings = [
        {"category": "HARM_CATEGORY_HARASSMENT", "threshold": "BLOCK_NONE"},
        {"category": "HARM_CATEGORY_HATE_SPEECH", "threshold": "BLOCK_NONE"},
        {"category": "HARM_CATEGORY_SEXUALLY_EXPLICIT", "threshold":
"BLOCK_NONE"},
        {"category": "HARM_CATEGORY_DANGEROUS_CONTENT", "threshold":
"BLOCK_NONE"},
    ]
    return genai.GenerativeModel(MODEL_NAME, safety_settings=safety_settings)

def translate_files():
    model = setup_gemini()
```

```
# Get list of files
if not os.path.exists(INPUT_FOLDER):
    print(f"Error: Folder '{INPUT_FOLDER}' not found.")
    return

files = [f for f in os.listdir(INPUT_FOLDER) if
os.path.isfile(os.path.join(INPUT_FOLDER, f))]
total_files = len(files)

print(f"Found {total_files} files to translate...")
print(f"Target Language: {TARGET_LANGUAGE}")
print(f"-----")

if not os.path.exists(OUTPUT_FOLDER):
    os.makedirs(OUTPUT_FOLDER)

for index, filename in enumerate(files):
    input_path = os.path.join(INPUT_FOLDER, filename)
    output_path = os.path.join(OUTPUT_FOLDER, filename)

    # 1. Skip if already done (Resume Capability)
    if os.path.exists(output_path):
        print(f"[{index+1}/{total_files}] Skipping {filename} (Already
exists)")
        continue

    print(f"[{index+1}/{total_files}] Translating {filename}...", end=" ",
flush=True)

    try:
        # 2. Read the file
        # Trying UTF-8 first, falling back to Latin-1 if it fails
        try:
            with open(input_path, "r", encoding="utf-8") as f:
                content = f.read()
        except UnicodeDecodeError:
            with open(input_path, "r", encoding="latin-1") as f:
                content = f.read()

        if not content.strip():
            print("Skipping (Empty file)")
            continue

        # 3. Construct the Prompt
        # Explicit instructions to prevent conversational filler
        prompt = (
```

```

        f"Translate the following text into {TARGET_LANGUAGE}. "
        f"Do not translate file paths, or the initial config section
(between --- ---), only the title section to express the translated title. "
        f"Maintain the original formatting strictly. "
        f"Output ONLY the translation, no introduction or conclusion.\n\n"
        f"---\n{content}\n---"
    )

    # 4. Call Gemini
    response = model.generate_content(prompt)

    # 5. Verify and Write Output
    if response.text:
        with open(output_path, "w", encoding="utf-8") as f:
            f.write(response.text)
            print("Done!")
    else:
        print("Failed (Model returned empty text - likely Safety Filter
triggered).")

    except Exception as e:
        print(f"Error: {e}")
        # Note: We continue to the next file even if this one fails

    # 6. Rate Limit Sleep
    # This is crucial for the free tier
    print(f"Waiting {DELAY_BETWEEN_FILES} seconds to respect rate limits...")
    time.sleep(DELAY_BETWEEN_FILES)

    print(f"-----")
    print("Batch translation complete.")

if __name__ == "__main__":
    translate_files()

```

Corrección del formato del bloque de metadatos

Para corregir posibles errores de formato en la estructura de los archivos, se desarrolló un script de reparación del bloque de metadatos. Su funcionamiento se basa en buscar la línea exacta donde comienza la etiqueta del título y reconstruir el archivo desde ese punto, asegurándose de insertar los tres guiones de apertura necesarios para el estándar YAML. Esto permite que, si un proceso previo dejó el encabezado mal formado o incompleto, el script lo restaure automáticamente para que la plataforma Wiki.js pueda indexar la página correctamente.

Python

```
import os

SOURCE_DIR = "wiki_es"
DEST_DIR = "wiki_temp"
SKIPPED = 0
PROCESSED = 0

os.makedirs(DEST_DIR, exist_ok=True)

for filename in os.listdir(SOURCE_DIR):
    src_path = os.path.join(SOURCE_DIR, filename)
    dest_path = os.path.join(DEST_DIR, filename)

    if not os.path.isfile(src_path):
        continue

    try:
        with open(src_path, "r", encoding="utf-8") as f:
            lines = f.readlines()

        # Check if file already starts correctly
        if len(lines) >= 2 and lines[0].startswith("---") and
lines[1].startswith("title"):
            # Write file unchanged
            with open(dest_path, "w", encoding="utf-8") as out:
                out.writelines(lines)
            continue

        # Find the first line that starts with "title"
        title_index = None
        for i, line in enumerate(lines):
            if line.startswith("title"):
                title_index = i
                break

        # If no title line is found, skip the file
        if title_index is None:
            print(f"Skipped (no title): {filename}")
            SKIPPED += 1
            continue

        # Create corrected content
        new_lines = ["---\n"] + lines[title_index:]

        with open(dest_path, "w", encoding="utf-8") as out:
            out.writelines(new_lines)
```



```
print(f"Fixed: {filename}")
PROCESSED += 1

except Exception as e:
    print(f"Error processing {filename}: {e}")
print(f"\nProcessing complete. Processed: {PROCESSED}, Skipped: {SKIPPED}")
```

Traspaso de la ruta de los archivos dentro de la wiki

Finalmente, se utiliza un script dedicado a la corrección de rutas internas para asegurar que la navegación entre páginas funcione en el nuevo entorno. El código emplea una expresión regular para localizar todos los enlaces internos dentro de los archivos Markdown y aplica una función de sustitución que cambia la antigua estructura de carpetas (típicamente bajo el prefijo "/wiki/") por la nueva jerarquía de la plataforma. Esto garantiza que todos los hipervínculos que conectan unos artículos con otros sigan operativos tras la migración.

```
Python
import os
import re
import shutil

# Configuración
ORIGEN = "C:/Users/Yago/BulkUp/Trabajo/Incibepedia/en/wiki"
DESTINO = "C:/Users/Yago/BulkUp/Trabajo/Incibepedia/en/wiki_remap"

# Regex para enlaces Markdown: [texto](url)
MD_LINK_REGEX = re.compile(r'\s*([^\s\]]+)\s*\]\s*\(\s*([^\s\)]+)\s*\)\s*')

archivos_modificados = 0
instancias_modificadas = 0

for root, dirs, files in os.walk(ORIGEN):
    for file in files:
        origen_path = os.path.join(root, file)

        # Mantener estructura de carpetas
        ruta_relativa = os.path.relpath(origen_path, ORIGEN)
        destino_path = os.path.join(DESTINO, ruta_relativa)
        os.makedirs(os.path.dirname(destino_path), exist_ok=True)

        # Solo procesar archivos Markdown
        if not file.lower().endswith(".md"):
            shutil.copy2(origen_path, destino_path)
```

```
        continue

    try:
        with open(origen_path, "r", encoding="utf-8") as f:
            contenido = f.read()

        cambios_en_archivo = 0

        def reemplazo(match):
            global cambios_en_archivo, instancias_modificadas
            texto, url = match.groups()

            if "/wiki/" in url:
                nueva_url = url.replace("/wiki/", "/")
                cambios_en_archivo += url.count("/wiki/")
                instancias_modificadas += url.count("/wiki/")
                return f"[{texto}](nueva_url)"

            return match.group(0)

        contenido_nuevo = MD_LINK_REGEX.sub(reemplazo, contenido)

        if cambios_en_archivo > 0:
            archivos_modificados += 1

        with open(destino_path, "w", encoding="utf-8") as f:
            f.write(contenido_nuevo)

    except UnicodeDecodeError:
        # Copiar tal cual si no es texto válido
        shutil.copy2(origen_path, destino_path)

    except Exception as e:
        print(f"Error procesando {origen_path}: {e}")

# Resumen final
print("Proceso completado.")
print(f"Archivos modificados: {archivos_modificados}")
print(f"Instancias reemplazadas: {instancias_modificadas}")
```